

Examen Parcial de IA

(5 de noviembre de 2021)

Duración: 90 min

1. (4 puntos) Deseamos construir un circuito integrado de manera que la intensidad de corriente necesaria por los diferentes elementos esté equilibrada en todo el circuito. Para simplificar el problema hemos supuesto que el circuito es una cuadrícula de $N \times M$. Los diferentes elementos que queremos colocar en el circuito son de K tipos diferentes y tenemos que colocar un número e_k de cada uno de ellos. Cada tipo de elemento necesita una intensidad de corriente específica.

Cada posición del circuito puede albergar P elementos de cualquier tipo y hemos de colocar todos los elementos.

El objetivo es conseguir que la suma de las intensidades que necesitan los elementos para cada fila y cada columna no supere un valor I y que la diferencia de intensidad de corriente entre una celda y cada una de sus cuatro vecinas contiguas no sea mayor que un valor V .

Se nos proponen las siguientes soluciones:

- a) Usar el algoritmo de Hill Climbing. El estado incluye, para cada componente (de los $C = \sum_{k \in K} e_k$ componentes), la coordenada en que se ubica, y estructuras adicionales que hagan falta para computar el resto de cosas de forma eficiente (como la intensidad total en una casilla según la asignación actual). Como solución inicial se construye una asignación vacía (toda pieza falta por asignar). Como operador usamos `colocar(pieza, fila, columna)`, que asigna la pieza a la posición $(fila, columna)$. La función heurística es:

$$h(n) = \sum_{i=1}^N \text{sobrecarga_fila}(i) + \sum_{j=1}^M \text{sobrecarga_columna}(j) + \sum_{i=1}^N \sum_{j=1}^M \text{sobrecarga_vecinas}(i, j) + W$$

donde `sobrecarga_fila(i)` y `sobrecarga_fila(j)` valen 1 si la suma de intensidades de la fila o columna superan I respectivamente, `sobrecarga_vecinas(i, j)` cuenta el número de diferencias de intensidad superiores a V entre la celda (i, j) y sus cuatro celdas vecinas, y W es el número de piezas todavía sin colocar.

A priori Hill Climbing no es adecuado para este problema porque no buscamos optimizar ninguna cantidad, si no llegar a una solución válida. En este caso, intentamos 'optimizar' el número de errores de la no solución, con intención de llegar a la solución, planteamiento que puede llegar a funcionar siempre que el modelo en su conjunto lleve hacia el espacio de soluciones.

Empezamos con una solución vacía, que es obviamente no solución. Se pueden proponer alternativas con todos los componentes ya colocados de forma aleatoria, pero no tiene sentido proponer que se coloquen de forma que se cumplan las restricciones (ya que entonces ya estaría el problema resuelto). En conjunto con el operador propuesto, podemos visitar todo el espacio de soluciones (aunque el heurístico no nos guíe hacia una solución, esta es visible).

El operador permite explorar todo el espacio de soluciones (y de no soluciones), y tiene un factor de ramificación de $C * N * M$ o de $K * N * M$ depende de como se interpreta, que es bastante alto dependiendo del problema de entrada. Particularmente, la propuesta de solución es ambigua: el operador colocar puede colocar un tipo de componente en un sitio (lo cual nos lleva a no poder mover componentes una vez puestos, y requeriría de condiciones de aplicación sobre cuantos se pueden colocar de cada tipo), o a colocar un componente concreto en una posición (permitiendo pues reubicar componentes, ya que un componente colocado puede volver a ser colocado con el operador -nada lo impide). Con la primera interpretación, colocar un componente que produce una sobrecarga en fila o columna nos lleva a no poder llegar a una solución directamente, mientras que con el otro, podríamos potencialmente llegar a una solución en el futuro. En ambos casos, falta comprobar que no coloquemos más de P componentes en una posición: o bien en la aplicabilidad del operador o en el heurístico como otra penalización. Un operador de intercambio entre piezas, teniendo en cuenta la ramificación de $C * (C - 1) / 2$, podría ser buena idea para mejorar la navegación por el espacio. No sería correcto, sin embargo, proponer cambiar el operador de colocar por uno de intercambio, ya que no podríamos ver todo el espacio de soluciones. De la misma forma, un operador de eliminar pieza no tiene mucho sentido, ya que con el heurístico dado, si algo se ha colocado (el heurístico ha bajado), rara vez quitarlo va a bajar el heurístico. El heurístico pondera de la misma forma añadir un componente que vulnerar otra restricción (excepto la de sobrecarga entre vecinos, que puede valer hasta 4 veces más), lo cual es provoca que no podamos colocar componentes que rompen restricciones (que parece a priori un comportamiento deseado).

Sin embargo, en algunos casos del problema, esto puede ser catastrófico. Por ejemplo, si la intensidad de todos los componentes es superior a V , no podríamos colocar ningún componente al comenzar, ya que vulneraría la potencia de los adyacentes (a pesar que podríamos colocar componentes de la misma intensidad a su lado en pasos posteriores, eliminando la sobrecarga). Un ejemplo de juguete que muestra esto sería colocar 2 componentes con intensidad $V + 1$ en un tablero de 2×1 : existe una solución obvia, pero el algoritmo jamás la encontraría. Por este mismo motivo, tener esta restricción como condición de aplicabilidad en lugar de en el heurístico sería nefasto. Precisamente es esta restricción sobre V la que genera la mayor cantidad de problemas con el modelo, particularmente generando picos de sierra como el que acabamos de comentar.

El problema de las restricciones en el heurístico podría ser solucionado añadiendo un ponderador a W , tal que sea mejor asignar componentes aunque se incumplan restricciones; y variar este ponderador en función de si las soluciones que nos devuelve el programa asignan todos los componentes o no.

El heurístico sigue teniendo un problema bastante claro, y es que es poco informativo. Dadas dos no-soluciones que incumplan la misma cantidad de restricciones, no sabemos cuál está más cerca de ser solución, y crea mesetas en la búsqueda. Esto se puede mejorar cambiando todos los componentes de la heurística de 'cuentas' a 'intensidades': por ejemplo, W de número de componentes no asignados a 'suma de intensidades de componentes no colocados': los componentes con intensidades más altas son los que más fácilmente vulnerarán restricciones, así que nos interesará colocarlos lo antes posible.

Potencialmente sería mejor penalizar cada una de las restricciones en función de cuánto sobrepasan el valor permitido. Es decir, `sobrecarga_fila(i)` sea por cuánta intensidad supera la fila el valor I , o mejor aún: el cuadrado de esta, para que el heurístico castigue mucho más sobrepasar el valor por mucho.

En resumen: la técnica podría funcionar bajo ciertas modificaciones, pero sería bastante frágil y no necesariamente devolvería una solución válida, potencialmente requiriendo de ajustes paramétricos en el heurístico para asegurar que funcione.

Una explicación menos exhaustiva puede llegar a puntuación completa, se ha aprovechado el espacio de esta solución para resolver problemas frecuentes vistos durante la corrección.

- b) Usar un algoritmo de programación de restricciones. Las variables son los diferentes elementos a colocar y los valores son las coordenadas (i, j) de la cuadrícula donde se podrían colocar. Las restricciones consideradas son que la suma de las intensidades de los elementos que compartan la misma fila y los que compartan la misma columna sea menor que I y que la diferencia entre la suma de las intensidades de los elementos de una celda con la suma de las intensidades de los elementos de las celdas vecinas contiguas no sea superior en V .

A priori, un algoritmo de programación de restricciones parece adecuado, ya que necesitamos encontrar una solución dentro de un espacio con muchas no soluciones. Un problema de asignación es muy indicado para este algoritmo.

En la representación actual tenemos un total de $C = \sum_{k \in K} e_k$ variables, una por componente; con un total de $N * M$ valores cada una. Una asignación en este espacio nos devuelve una asignación de componentes a posiciones, que es lo que buscamos. No sería posible, por ejemplo, una representación de K variables, ya que no sabemos donde va cada componente de su tipo (esto ha sido penalizado).

Las restricciones descritas en la propuesta requieren de razonar sobre intensidades de componentes en posiciones particulares. En la representación actual, la única manera de comprobar esto es sumar la intensidad de todas las variables con valor asignado relativo a la fila, columna, o celda (en función de cada restricción). Ya que no sabemos donde se coloca cada componente hasta su asignación, esto hace que todas las restricciones sean C-arias, lo cual es bastante ineficiente.

A nivel de corrección, faltaría una restricción para asegurarse de que no colocamos más de P componentes en una misma celda, que en este modelado sería una restricción C-aria como el resto, o un conjunto de $\binom{C}{P+1}$ restricciones $(P+1)$ -arias (en grupos de $P+1$, asegurar que todas las $P+1$ variables no compartan la misma celda). En ningún caso, para esta representación, hace falta comprobar si un componente se coloca dos veces, o si un componente falta por colocar: satisfacción de restricciones da un valor a cada variable.

Una forma sencilla mejorar ligeramente la complejidad del problema sería dividir las variables de celda de componente $C[x] = (i, j)$ a dos variables $C_fila[x] = i$, $C_col[x] = j$, que si bien aumenta el número de variables, reduce el número de posibles valores de cada una y simplifica la complejidad de las restricciones de fila y columna (ya que ahora las variables tienen N y M valores), pero no mejora el resto de restricciones (se puede argumentar que no las empeora porque el número de valores sigue siendo el mismo). Otra mejora bastante clave sería introducir más variables que indiquen, para cada celda, la intensidad acumulada en ellas (que simplifica radicalmente la comprobación de las restricciones del enunciado, pero esconde la complejidad en restricciones para computar cuánta intensidad hay en una celda en $N*M$ restricciones $C+1$ -arias).

Sin embargo, lo que mejora más el modelado sería invertir las variables y los valores, es decir: cada variable representa el componente ubicado en una de las P posiciones de cada celda: $P * N * M$ variables con valores $K \cup \{\emptyset\}$, o $C = \sum_{k \in K} e_k \cup \{\emptyset\}$, donde $\emptyset$ representa la no-asignación de un componente a una posición. Este valor adicional sería extremadamente importante, ya que si no estaríamos forzando al algoritmo a saturar el circuito por completo (toda celda llena hasta P componentes). En el primer caso, cada variable contiene el tipo de componente, y en el segundo, el identificador de un componente concreto. Una propuesta alternativa a ambas sería tener $N * M$ variables, con valores 'los componentes colocados en ella, hasta P '. En esta propuesta, el número de valores sería $\sum_{k=0}^P \binom{C}{k}$, lo cual la hace mucho menos preferible comparado a la propuesta anterior (aunque razonable si P es muy bajo).

Además, de forma similar a lo que se proponía anteriormente, podemos añadir $N*M$ variables para representar la intensidad de cada celda (ahora sin tanta complejidad), y pasaríamos a tener las restricciones siguientes:

- Para cada celda, la intensidad de la celda es la suma de intensidades de los componentes que no son la no-asignación ($N*M$ restricciones $(P+1)$ -arias).

- Para cada 2 celdas contiguas, la diferencia de intensidad no supera V ($M^*(N-1)+N^*(M-1)$ restricciones binarias)
- Para cada fila y columna, la suma de intensidades no supera I (N y M restricciones M -arias y N -arias respectivamente).
- Todo componente debe estar colocado:
 - Caso $K \cup \{\emptyset\}$: Restricción N^*M^*P -aria para comprobar que el número de componentes de cada tipo es exactamente el del enunciado. Alternativamente, K restricciones que comprueben lo mismo para cada tipo de componente (igual pero descompuesto).
 - Caso $C = \sum_{k \in K} e_k \cup \{\emptyset\}$: N^*M^*P restricciones binarias de desigualdad (ignorando $\emptyset$) para asegurar que cada componente esté colocado máximo 1 vez, y una restricción N^*M^*P -aria para comprobar que se asignen C componentes. Alternativamente, se puede hacer con una biyección con un conjunto de variables como el propuesto (C variables asociadas a posición), de forma que todo componente C está asociado a una posición (mediante del orden de $C^*(N^*M^*P)$ restricciones binarias), y por tanto colocado (que aunque puede verse como algo bastante ineficiente, puede funcionar bien).

En resumen, la técnica propuesta es adecuada y obtiene una solución válida, pero los costes de restricciones en la versión original hacen que sea bastante ineficiente (mejorable). Un modelado inverso como el descrito bajará radicalmente el coste algorítmico, y sería mucho más deseable. Una explicación menos exhaustiva de lo descrito es suficiente para obtener puntuación completa. Otros tipos de mejoras se han dado como parcialmente válidas en función de cuánto mejoran el algoritmo.

Comenta cada una de las soluciones que se proponen, analizando si la técnica escogida es adecuada para este problema, si cada uno de los elementos de la solución son correctos o no (cada uno por separado y en conjunción los unos con los otros). Incluye un análisis de los costes algorítmicos y/o factores de ramificación allá donde sea necesario. Justifica tu respuesta.

2. (6 puntos) La empresa *Cloud For U* se dedica a revender tiempo de cómputo para cloud computing y para ello ha montado un servicio que recibe solicitudes para usar ese tiempo. Este servicio es un sistema de pre-reserva que nos permite saber las solicitudes para un mes completo. Cada petición nos indica un día y hora de inicio y un tiempo de uso en horas, indicando además la prioridad de la petición (de 1 a 3, siendo 1 la menos prioritaria). Una petición ha de ejecutarse completamente, sin interrupciones y usando exclusivamente una CPU. *Cloud For U* compra el tiempo de cómputo en una CPU durante un mes a M euros y cobra el tiempo de cómputo a $\text{prioridad} \times P$ euros/hora, con el compromiso de que si no se sirve la petición, ésta se servirá seguro el mes siguiente a solo P euros/hora independientemente de su prioridad inicial.

La empresa nos plantea saber cuantas CPUs debe alquilar en el mes actual para poder obtener el mayor beneficio posible, intentando reducir el número de peticiones diferidas al mes siguiente.

En los siguientes apartados se proponen diferentes alternativas para algunos de los elementos necesarios para plantear la búsqueda (solución inicial, operadores, función heurística,...). Para cada una de ellas se ha de comentar la solución que se propone, analizando si la técnica escogida es adecuada para este problema, si cada uno de los elementos de la solución son correctos o no (cada uno por separado y en conjunción los unos con los otros). Incluye un análisis de los costes algorítmicos y/o factores de ramificación allá donde sea necesario. Justifica tu respuesta.

- a) Se plantea aplicar Hill-climbing, para crear la solución inicial suponemos que necesitamos tantas CPUs como peticiones nos han llegado, de manera que asignamos cada petición a una CPU diferente. Como operador usamos fusionar dos CPUs, que consiste en combinar las peticiones de dos CPUs en una sola eliminando la CPU sobrante. Como función heurística consideraremos que alquilaremos las CPUs en la solución que tengan más del 90 % de ocupación ($CPU90$), por lo tanto minimizaremos la función:

$$h(n) = (N_{CPU} \times M) - \sum_{\forall p_i \in CPU90} prio(p_i) \times P \times long(p_i) + \lambda \times \sum_{\forall p_i \in \overline{CPU90}} P \times long(p_i)$$

donde N_{CPU} es el número de CPUs usadas en la solución, $CPU90$ es el conjunto de CPUs ocupadas a más del 90 % y $\overline{CPU90}$ es su conjunto complementario, $long(p_i)$ es la longitud en horas de la petición y $prio(p_i)$ es la prioridad de la petición. λ es un peso para ajustar la penalización de las tareas que se diferirán al mes siguiente.

El planteamiento del problema como una búsqueda en el espacio de soluciones es correcto, buscamos minimizar el número de CPUs para así maximizar el beneficio.

La solución inicial se encuentra dentro del espacio de soluciones y se puede construir con un coste muy bajo ($O(n_{Peticiones})$, donde $n_{Peticiones}$ es el número total de peticiones que hemos recibido). Sin embargo la calidad de la solución es muy mala. Una generación de solución inicial mejor con el mismo coste lineal sería ir rellenando CPU's con peticiones en el orden de llegada, y crear una nueva CPU cuando la petición ya no cabe en la CPU actual sin colisionar con otra ya existente. Si se quisiera ejecutar Hill Climbing varias veces deberíamos añadir algo de aleatoriedad en el generador de la solución inicial.

El operador propuesto sería correcto si comprobara antes que no se van a producir colisiones/solapamientos entre trabajos al fusionar las peticiones de dos CPUs. Su factor de ramificación es $(N_{CPU})^2$. Aunque no nos permite mover peticiones de CPU y por lo tanto no nos permite desde una solución acceder a todas las otras soluciones, la combinación de este operador con la solución inicial de una petición por CPU si que permite en teoría explorar potencialmente todo el espacio de soluciones desde esa solución inicial. Eso si, seguramente si probáramos el operador de forma empírica, veríamos que tiene dificultad para aplicarse hacia el final de la exploración, con las CPU's bastante llenas, y es muy probable que (con el heurístico planteado) difícilmente llegaran a ocuparse al 90 %. Por ello es recomendable añadir un operador de mover petición de una CPU a otra para intentar optimizar el uso de cada CPU.

El heurístico no calcula el coste-beneficio exacto, sino que se opta por penalizar las peticiones que se dejan por servir para el mes siguiente. El primer elemento del heurístico es el gasto en CPU's, el segundo elemento es lo que se gana con las asignaciones que se sirven en el mes, y el tercer elemento penaliza las peticiones servidas el mes siguiente con un factor λ arbitrario, independientemente de la prioridad de la tarea. Se podría intentar ajustar mejor el coste de las peticiones que se dejan para el mes siguiente calculando lo que dejaríamos de cobrar: $\sum_{\forall p_i \in \overline{CPU90}} (prio(p_i) - 1) \times P \times long(p_i)$, pero esto no resolvería el problema principal del heurístico: no penaliza las no soluciones. Además, debido al límite arbitrario del 90 % de ocupación de una CPU para decidir que esas peticiones se sirven este mes, pasa una cosa curiosa: muy probablemente en la solución inicial la mayoría de peticiones estén por debajo del 90 % de ocupación, y por lo tanto con el heurístico propuesto pertenecen al conjunto $\overline{CPU90}$ en el que no se tiene en cuenta la prioridad de las peticiones asignadas en una CPU, y no se tendrá en cuenta hasta que se supera el 90 % de ocupación en esa CPU y pasen al conjunto $CPU90$. Por lo tanto es probable que la mayor parte del tiempo la búsqueda no esté guiada por las prioridades de las peticiones.

- b) Se plantea usar algoritmos genéticos, como representación escogemos una cadena de bits donde para cada petición se tiene un número en binario que es la CPU que tiene asignada; suponemos que ese número es un valor dentro del rango $(NP_3 \dots NC + 1)$, donde NP_3 es el número de peticiones de prioridad 3 y NC es el número total de CPU's, añadimos un valor adicional para representar una CPU ficticia (a la que se asignan

las peticiones diferidas al mes siguiente). Para generar una solución se van generando al azar valores entre los límites mencionados y se asignan consecutivamente a las peticiones de la siguiente manera, con un 50% de probabilidad se le asigna el valor de una CPU real y con un 50% de probabilidad se le asigna la CPU ficticia. Utilizamos los operadores habituales de cruce y mutación. Como función heurística consideraremos que alquilaremos las CPUs en la solución que no sean la ficticia, por lo tanto minimizaremos la función:

$$h(n) = (N_{CPU} \times M) - \sum_{\forall p_i \notin CPU_f} prio(p_i) \times P \times long(p_i) + \sum_{\forall p_i \in CPU_f} P \times long(p_i)$$

donde N_{CPU} es el número de CPUs que no son la ficticia y que tienen peticiones asignadas, CPU_f es el conjunto de peticiones que están en la CPU ficticia, $long(p_i)$ es la longitud en horas de la petición y $prio(p_i)$ es la prioridad de la petición.

El planteamiento del problema como una búsqueda en el espacio de soluciones a través de un algoritmo genético es correcto, buscamos la solución que minimice costes y maximice beneficios.

La codificación permite representar la asignación de peticiones a CPU's, pero es capaz de codificar CPU's que no son ni las reales ni la ficticia, por lo que fácilmente nos saldremos del espacio de soluciones. Sin embargo en este caso esto tiene una solución muy sencilla, declarando toda CPU que no existe como una CPU ficticia, de forma que toda asignación de una petición a una CPU que no existe no se serviría. Otro problema de la codificación inicial es la arbitrariedad del rango de valores que codifican CPU's reales, que no permite representar todas las soluciones posibles y podría ser insuficiente para poder servir la mayoría de peticiones.

La generación de soluciones iniciales se ha planteado correctamente al tener una aleatoriedad que es capaz de crear numerosos individuos para la población inicial de soluciones, pero al no comprobar si se producen solapamientos entre las peticiones asignadas a una CPU real es muy probable que genere no soluciones. El coste de generación es $O(NP \times NI)$, siendo NP el número de peticiones y NI el número de individuos de cada generación del algoritmo. La calidad de las soluciones es entre media y baja, ya que más o menos la mitad de las peticiones se diferirán al mes siguiente.

Los operadores de cruce y mutación estandar no comprueban ninguna restricción, por lo que muchas veces convertirán una solución en una no solución:

- el primer problema es que ambos operadores pueden generar una colisión entre dos peticiones asignadas a una misma CPU;
- el segundo problema es que ambos operadores pueden generar un número de CPU fuera del rango de CPU's reales + la ficticia. Este segundo problema se puede solucionar si consideramos todas las CPU's no reales como ficticias.

La función heurística tiene tres problemas. El primero es que el operador ni controla ni penaliza soluciones que están fuera del espacio de soluciones. Suponiendo que hemos resuelto el problema de las CPU's ficticias como todas las codificaciones que no son una CPU real, el heurístico debería penalizar las colisiones entre peticiones asignadas a una misma CPU real en función del número de días de solapamiento (a mayor solapamiento, peor solución), pero dando peso sobretudo a la existencia de solapamiento (un solapamiento de un solo día ya es un problema). El segundo es que no solo no se calcula bien la pérdida de beneficio provocada por diferir peticiones al mes siguiente, sino que tenemos todas las peticiones diferidas como penalización y sin un factor que pondere el efecto de esta penalización. Y el tercero es que en principio un algoritmo genético intenta maximizar la función de evaluación, entendida como función de *fitness*, por lo que en este caso habría que girar la función (beneficios-costes).

- c) Usar el algoritmo de A*. Definimos el estado como la asignación de peticiones a CPU's. Partimos de la asignación vacía en la que hay cero CPU's y cero peticiones asignadas. Tendremos un operador para añadir una CPU (comprueba que no estuviera ya en la solución), el coste del operador es M . Tendremos otro operador para añadir una petición a una CPU (comprueba que la petición no haya sido añadida a la solución, y que la CPU ya ha sido añadida previamente con el otro operador), su coste es $-prio(p_i) \times P \times long(p_i)$, donde $long(p_i)$ es la longitud en horas de la petición y $prio(p_i)$ es la prioridad de la petición p_i que colocamos. La función heurística es la suma de las longitudes en horas de las peticiones que nos quedan por colocar en CPU's, expresada así:

$$h(n) = \sum_{\forall p_i \notin P_{assign}} long(p_i)$$

donde P_{assign} es el conjunto de peticiones ya asignadas.

El algoritmo A* está pensado para encontrar la solución óptima. Si entendemos que el problema nos pide encontrar la configuración que obtiene el beneficio máximo con el mínimo número de peticiones diferidas al mes siguiente, a priori sería el algoritmo adecuado para ello. Si entendiéramos que solo nos pide maximizar en lo posible el beneficio y minimizar tareas diferidas sin encontrar el óptimo, el A* no justificaría su coste elevado para la resolución del problema, ya que introducimos una complejidad algorítmica innecesaria. Se plantea el problema como un espacio de estados en el que partimos de un estado inicial vacío (correcto), y

se va construyendo una solución con los operadores, que van añadiendo CPU's y peticiones a las CPU's (un planteamiento en principio correcto). El operador para añadir CPU's es correcto, comprueba que la CPU no estuviera ya en la solución. El factor de ramificación es $O(N_{CPU})$ si distinguimos entre las diferentes CPUs a la hora de escoger una más a contratar, o bien es $O(1)$ si nos da lo mismo que CPU añadimos de la solución (una cualquiera que aún no esté en la solución). El coste M se irá agregando dentro de la función $g(n)$, que el algoritmo de A^* intentará minimizar, lo cual es correcto.

El operador para añadir peticiones no es correcto. Aunque comprueba que la tarea no haya sido añadida ya y que la CPU exista, no comprueba que la tarea no se solape con otra tarea ya asignada en esa CPU. El factor de ramificación es $O(NP \times N_{CPU})$, siendo NP el número de peticiones a asignar. El coste $-prio(p_i) \times P \times long(p_i)$ puede parecer que tenga sentido (reducimos los gastos de las CPU's con los beneficios obtenidos) en realidad es incorrecto, ya que A^* no nos puede asegurar optimalidad si los costes que se van acumulando en la función $g(n)$ pueden ser negativos.

La función heurística $h(n)$ propuesta en este problema nos da una idea de las peticiones que quedan por asignar. El problema es que esta función no es admisible, ya que cuando tuviéramos la asignación óptima de peticiones al mes actual la función $h(n)$ no tendría valor cero, nos diría que tenemos aun peticiones por asignar, sobreestimando la distancia a la solución óptima y forzándonos a asignarlas todas en el mes actual. Además, mientras que $g(n)$ calcula dinero (costes - beneficios), $h(n)$ intenta calcular tiempo de cálculo por asignar, por lo que la suma $g(n) + h(n)$ que A^* intentará minimizar tiene sumandos con unidades diferentes.

Las notas se publicarán el día **26 de noviembre**.